110. Balanced Binary Tree Easy

Acceptance: 56.9% | LeetCode

Given a binary tree, determine if it is **height-balanced**.

Example 1:

```
**Input:** root = [3,9,20,null,null,15,7]
**Output:** true
```

Example 2:

```
**Input:** root = [1,2,2,3,3,null,null,4,4]
**Output:** false
```

Example 3:

```
**Input:** root = []
**Output:** true
```

Constraints:

- The number of nodes in the tree is in the range `[0, 5000]`.
- `-104 <= Node.val <= 104`

Claude Opus Solution

Balanced Binary Tree

1. Intuition

Definition: A tree is height-balanced if, for **every** node, the heights of its left and right subtrees differ by at most 1.

Key Insight: The naive way computes height separately for every node — but computing height already requires traversing the entire subtree. If we compute height *and* check balance in the **same traversal**, we avoid redundant work.

The trick is a **bottom-up DFS**: as we compute each node's height, we simultaneously verify balance. We use a **sentinel value (-1)** to signal "this subtree is unbalanced," which propagates up and short-circuits the entire computation.

2. Approach

```
function height(node):
    if node is null:
        return 0                      # empty subtree has height 0

    left = height(node.left)          # recurse left
    if left == -1: return -1          # left subtree already unbalanced → propagate

    right = height(node.right)        # recurse right
    if right == -1: return -1         # right subtree unbalanced → propagate

    if abs(left - right) > 1:
        return -1                     # THIS node violates balance → signal failure

    return max(left, right) + 1       # otherwise return actual height

isBalanced(root):
    return height(root) != -1
```

3. Python Solution

Optimal: Bottom-Up DFS (O(n))

```python
from typing import Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        # We use -1 as a sentinel meaning "unbalanced subtree found".
        # If height() ever returns -1, the whole tree is unbalanced.
```

```
        return self._height(root) != -1

    def _height(self, node: Optional[TreeNode]) -> int:
        # Base case: empty subtree has height 0 and is trivially balanced.
        if node is None:
            return 0

        # Compute left subtree height (bottom-up).
        left = self._height(node.left)
        # Short-circuit: if left subtree is already unbalanced,
        # no need to check further — propagate the failure signal upward.
        if left == -1:
            return -1

        # Compute right subtree height.
        right = self._height(node.right)
        if right == -1:
            return -1

        # Check the balance condition AT THIS NODE.
        # If the height difference exceeds 1, mark as unbalanced.
        if abs(left - right) > 1:
            return -1

        # This subtree is balanced; return its actual height.
        # Height = 1 (this node) + the taller of the two children.
        return max(left, right) + 1
```

Why this beats the reference top-down solution

The reference `solution.py` calls `find_depth` for each node and *separately* recurses with `isBalanced`. Even with memoization, the structure is harder to reason about. The `my_solution.py` is essentially the same bottom-up idea as mine (returning `(bool, height)` tuples), which is also clean and correct.

---

4. Complexity Analysis

Approach	Time	Space
Bottom-up DFS (optimal)	**O(n)**	**O(h)**
Naive top-down DFS	$O(n^2)$	O(h)

Optimal — Time O(n): Each node is visited exactly **once** during the single post-order traversal. Height and balance are computed together, so there is no repeated work.

Optimal — Space O(h): The recursion stack depth equals the tree height `h`. - Balanced tree: `h = O(log n)` - Skewed (worst case) tree: `h = O(n)` — but a deeply skewed tree gets caught as unbalanced early, though the stack can still grow to that depth before detection.

Why naive is $O(n^2)$: The naive approach computes `find_depth` (an O(n) operation) at *every* node, giving O(n × n). For a skewed tree, this becomes quadratic. Memoization (as in the reference) reduces it back to O(n) but at the cost of extra O(n) hash-table space.

---

5. Alternative Approaches

a) Top-Down DFS (naive, $O(n^2)$) For each node, independently compute left/right subtree heights and check balance, then recurse into children. Simple to write but recomputes heights repeatedly → quadratic time.

b) Top-Down + Memoization (O(n) time, O(n) space) The reference `solution.py`. Cache each node's height in a hash table to avoid recomputation. Correct and O(n), but uses extra O(n) space and is less elegant than the single-pass bottom-up method.

c) Iterative Post-Order Traversal Replace recursion with an explicit stack to avoid recursion-depth limits (relevant for very deep trees near the 5000-node constraint in some languages). Same O(n) time, more complex code.

Summary

The winning pattern here is **combining two computations into one traversal**: instead of computing height and checking balance separately, do both bottom-up and use a **sentinel value (-1)** to short-circuit and propagate failure. This converts an $O(n^2)$ problem into a clean O(n) solution.

Codex Solution

1. Intuition

A binary tree is **height-balanced** if, for **every node**, the heights of its left and right subtrees differ by at most `1`.

So for each node, we need two pieces of information:

1. Is the left subtree balanced?
2. Is the right subtree balanced?
3. What are the heights of the left and right subtrees?

A very efficient pattern here is **bottom-up DFS**.

Instead of repeatedly computing subtree heights from scratch, we compute the height of each subtree once while checking balance at the same time.

The key idea:

> For each node, return its height if it is balanced.
> If it is not balanced, return a special value such as `-1`.

This allows us to stop early when we find an unbalanced subtree.

---

2. Approach

We define a helper function:

```
height(node):
    if node is None:
        return 0

    left_height = height(node.left)
    if left_height == -1:
        return -1

    right_height = height(node.right)
    if right_height == -1:
        return -1

    if abs(left_height - right_height) > 1:
        return -1

    return max(left_height, right_height) + 1
```

Then:

```
isBalanced(root):
    return height(root) != -1
```

Why does this work?

- A `None` node has height `0`.
- A leaf node has height `1`.
- If either child subtree is unbalanced, the current subtree is also unbalanced.
- If the current node's left and right subtree heights differ by more than `1`, it is unbalanced.
- Otherwise, return the height of the current subtree.

---

3. Python Solution

```python
from typing import Optional

# Definition for a binary tree node.
# LeetCode provides this class automatically.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right


class Solution:
    def isBalanced(self, root: Optional['TreeNode']) -> bool:
        """
        Returns True if the binary tree is height-balanced.

        A tree is balanced if for every node, the height difference between
        its left and right subtree is at most 1.
        """

        def get_height(node: Optional['TreeNode']) -> int:
            """
            Returns the height of the subtree rooted at `node`.

            If the subtree is not balanced, return -1 as a signal.
            """

            # Base case:
            # An empty subtree has height 0 and is balanced.
            if node is None:
                return 0

            # Recursively compute the height of the left subtree.
            left_height = get_height(node.left)

            # If the left subtree is already unbalanced,
            # propagate -1 upward immediately.
            if left_height == -1:
                return -1

            # Recursively compute the height of the right subtree.
            right_height = get_height(node.right)

            # If the right subtree is already unbalanced,
            # propagate -1 upward immediately.
            if right_height == -1:
                return -1

            # Check whether the current node is balanced.
            # If the height difference is greater than 1,
            # this subtree is unbalanced.
            if abs(left_height - right_height) > 1:
                return -1

            # Otherwise, the subtree is balanced.
            # Its height is 1 plus the taller child subtree.
            return max(left_height, right_height) + 1

        # If get_height(root) returns -1, the tree is unbalanced.
        # Otherwise, it returns a valid height, meaning the tree is balanced.
        return get_height(root) != -1
```

---

4. Complexity Analysis

Let `n` be the number of nodes in the tree, and let `h` be the height of the tree.

Time Complexity: `O(n)`

Each node is visited at most once.

At every node, we do constant work:

- compute left height
- compute right height
- compare their difference
- return height or `-1`

Therefore, the total time complexity is:

```
O(n)
```

Space Complexity: `O(h)`

The extra space comes from the recursion call stack.

- In a balanced tree, the height is `O(log n)`, so stack space is `O(log n)`.
- In a completely skewed tree, the height is `O(n)`, so stack space is `O(n)`.

Therefore, the general space complexity is:

```
O(h)
```

Worst case:

```
O(n)
```

---

5. Alternative Approaches

Alternative 1: Top-Down DFS

For each node:

1. Compute the height of the left subtree.
2. Compute the height of the right subtree.
3. Check if their difference is at most `1`.
4. Recursively check if the left and right subtrees are balanced.

Example idea:

```python
def height(node):
    if node is None:
        return 0
    return max(height(node.left), height(node.right)) + 1

def isBalanced(root):
    if root is None:
        return True

    left = height(root.left)
    right = height(root.right)

    return (
        abs(left - right) <= 1
        and isBalanced(root.left)
        and isBalanced(root.right)
    )
```

However, this can be inefficient because the same subtree heights may be recomputed many times.

Time complexity:

```
O(n^2)
```

in the worst case for a skewed tree.

Alternative 2: Top-Down DFS with Memoization

We can store the height of each node in a hash map so each height is computed once.

This improves the time complexity to:

```
O(n)
```

But it uses extra space for the hash map:

```
O(n)
```

The bottom-up DFS solution is usually cleaner because it avoids the need for memoization.